■ PYTHON
Interview Questions & Answers

For Freshers — Simple English, Easy to Remember

60+ Most-Asked Python Questions with Clear Answers + Real Examples

SECTION 1 — Python Basics

Q1. What is Python? Why is it used in Data Analytics?

Answer: Python is a simple, easy-to-read programming language. In data analytics, it is used to clean data, analyse data, build charts, and create machine learning models. It has many ready-made tools (libraries) like Pandas, NumPy, and Matplotlib that make data work easy.

Q2. What is the difference between a List, Tuple, Set, and Dictionary?

Answer: List [ ] — ordered, can change, allows duplicates. Example: [1,2,3]. Tuple ( ) — ordered, CANNOT change, allows duplicates. Example: (1,2,3). Set { } — unordered, no duplicates. Example: {1,2,3}. Dictionary { } — stores key-value pairs. Example: {'name':'Rahul', 'age':22}.

Example: Use List when order matters and data changes. Use Tuple for fixed data. Use Set to remove duplicates. Use Dictionary for labelled data.

Q3. What is the difference between mutable and immutable?

Answer: Mutable = can be changed after creation. Examples: list, dictionary, set. Immutable = CANNOT be changed after creation. Examples: tuple, string, integer.

Example: a = [1,2,3]; a[0] = 10 — works (list is mutable). b = (1,2,3); b[0] = 10 — gives ERROR (tuple is immutable).

Q4. What is a variable in Python?

Answer: A variable is a name that stores a value. Python automatically figures out the data type — you don't need to declare it.

Example: name = 'Priya' / age = 25 / salary = 45000.50 / is_active = True

Q5. What are the basic data types in Python?

Answer: int (whole numbers: 1, 25, -10), float (decimal numbers: 3.14, 99.5), str (text: 'Hello'), bool (True or False), None (no value / empty).

Q6. What is the difference between == and = ?

Answer: = is assignment — it gives a value to a variable. == is comparison — it checks if two values are equal and returns True or False.

Example: x = 5 (assign 5 to x). x == 5 (check if x equals 5 → True).

Q7. What are if-else statements?

Answer: If-else lets your code make decisions. If the condition is True, run one block of code. Otherwise (else), run another block.

Example: if marks >= 50: print('Pass') else: print('Fail')

Q8. What is a for loop and while loop?

Answer: For loop repeats code for each item in a list or range. While loop repeats code as long as a condition is True.

Example: for i in range(5): print(i) — prints 0,1,2,3,4

Q9. What is a function in Python?

Answer: A function is a block of reusable code. You define it once and call it many times. Use 'def' keyword to create a function.

Example: def add(a,b): return a+b / result = add(3,4) — result is 7

Q10. What is a lambda function?

Answer: A lambda function is a small, one-line anonymous function (no name). Used for quick, simple operations.

*Example: square = lambda x: x*x / square(5) = 25*

Remember: Lambda = mini function in one line

SECTION 2 — Pandas (Most Important for Data Analysis)

Q11. What is Pandas? Why is it important?

Answer: Pandas is a Python library for working with data in tables. It has two main structures: Series (single column, like a list) and DataFrame (full table with rows and columns, like Excel). It's the most important tool for data analysis in Python.

Example: import pandas as pd / df = pd.read_csv('data.csv')

Q12. What is a DataFrame and a Series?

Answer: Series = one column of data with an index. Like a single column in Excel. DataFrame = a full table with multiple rows and columns. Like a full Excel sheet.

Example: s = pd.Series([10,20,30]) / df = pd.DataFrame({'Name':['A','B'], 'Age':[22,25]})

Q13. How do you read a CSV file in Pandas?

Answer: Use pd.read_csv(). You can also read Excel with pd.read_excel(), JSON with pd.read_json(), and from database with pd.read_sql().

Example: df = pd.read_csv('students.csv') / df.head() — shows first 5 rows

Q14. What are the first things you do after loading data?

Answer: df.shape — rows and columns count. df.head() — first 5 rows. df.tail() — last 5 rows. df.info() — column names, data types, non-null counts. df.describe() — statistics (mean, min, max, etc.). df.isnull().sum() — count missing values per column.

Remember: Always explore data first before doing anything.

Q15. What is the difference between loc and iloc?

Answer: loc uses LABELS (column names, row labels) to select data. iloc uses NUMBERS (0,1,2... positions) to select data.

Example: df.loc[0,'Name'] — row 0, column 'Name'. df.iloc[0,1] — row 0, column at position 1.

Remember: loc = label, iloc = integer position

Q16. How do you handle missing values (NaN) in Pandas?

Answer: Check missing values: df.isnull().sum(). Drop rows with missing: df.dropna(). Fill with a value: df.fillna(0) or df.fillna('Unknown'). Fill with column average: df['salary'].fillna(df['salary'].mean()).

Remember: Don't just delete NaN! First understand why it's missing.

Q17. How do you filter rows in a DataFrame?

Answer: Use conditions inside square brackets.

Example: df[df['marks'] > 80] — students with marks > 80. df[(df['city']=='Mumbai') & (df['age']>25)] — multiple conditions with & (and) or | (or).

Q18. How do you add a new column?

Answer: Simply assign a value to a new column name.

Example: df['total'] = df['maths'] + df['science'] — adds maths + science into new column 'total'.

Q19. What is groupby() in Pandas?

Answer: groupby() groups rows by a column and lets you do calculations on each group. It follows Split → Apply → Combine logic.

Example: df.groupby('department')['salary'].mean() — average salary per department.

Remember: groupby = GROUP BY in SQL

Q20. What is merge() in Pandas?

Answer: merge() joins two DataFrames like SQL JOINs. You specify which column to join on and what type of join (inner, left, right, outer).

Example: merged = pd.merge(df1, df2, on='StudentID', how='left')

Remember: merge() = JOIN in SQL

Q21. What is the difference between concat() and merge()?

Answer: concat() stacks DataFrames vertically (adds more rows) or horizontally (adds more columns). merge() joins DataFrames based on a common column — like a database JOIN.

Example: pd.concat([df1,df2]) — stacks two tables. pd.merge(df1,df2,on='id') — joins on id.

Q22. What is apply() function?

Answer: apply() runs a function on each row or column of a DataFrame. Useful for custom transformations.

Example: df['grade'] = df['marks'].apply(lambda x: 'A' if x>=90 else 'B' if x>=80 else 'C')

Q23. How do you rename columns?

Answer: Use df.rename(columns={'old_name':'new_name'})

Example: df.rename(columns={'Sal':'Salary', 'Emp':'Employee'}, inplace=True)

Q24. How do you remove duplicates?

Answer: df.duplicated() — finds duplicate rows. df.drop_duplicates() — removes duplicate rows. You can specify columns: df.drop_duplicates(subset=['email']).

Q25. How do you sort a DataFrame?

Answer: df.sort_values('column_name') for ascending. df.sort_values('column_name', ascending=False) for descending. df.sort_values(['col1','col2']) for multiple columns.

Example: df.sort_values('salary', ascending=False) — highest salary first.

Q26. What is pivot_table in Pandas?

Answer: pivot_table creates a summary table — like Excel's pivot table. You can group by rows and columns and apply aggregate functions.

Example: df.pivot_table(values='sales', index='region', columns='month', aggfunc='sum')

Q27. How do you change data types in Pandas?

Answer: Use astype(). Common conversions: df['age'].astype(int), df['date'].astype('datetime64'), pd.to_datetime(df['date']), pd.to_numeric(df['price']).

Remember: Wrong data types cause wrong calculations — always check!

SECTION 3 — NumPy

Q28. What is NumPy?

Answer: NumPy is a Python library for fast mathematical operations on arrays (lists of numbers). It is the foundation of Pandas. Operations on NumPy arrays are much faster than regular Python lists.

Example: import numpy as np / arr = np.array([1,2,3,4,5])

Q29. What is the difference between a Python list and a NumPy array?

Answer: Python list can hold mixed types (numbers, strings, etc.) and is slow for math. NumPy array holds only one type and is very fast for mathematical operations (10-100x faster).

*Example: np.array([1,2,3]) * 2 = [2,4,6] — all elements multiplied at once.*

Q30. What is Broadcasting in NumPy?

Answer: Broadcasting allows operations between arrays of different sizes. NumPy automatically expands the smaller array to match the bigger one.

Example: arr = np.array([1,2,3]) / arr + 10 = [11,12,13] — 10 is added to each element.

Remember: Broadcasting = apply one value to all elements automatically

Q31. What are some commonly used NumPy functions?

Answer: np.mean() — average. np.median() — middle value. np.std() — standard deviation. np.sum() — total. np.min() / np.max() — smallest/biggest. np.zeros() / np.ones() — create arrays. np.arange() — like range(). np.reshape() — change shape of array.

SECTION 4 — Data Visualisation

Q32. What are the main Python libraries for charts/graphs?

Answer: Matplotlib — the base library, most customisable. Seaborn — built on Matplotlib, makes beautiful statistical charts easily. Plotly — for interactive charts. Use Seaborn for analysis, Plotly for dashboards.

Example: import matplotlib.pyplot as plt / import seaborn as sns

Q33. What chart do you use for what type of data?

Answer: Bar chart — compare categories (sales by region). Line chart — show trends over time (monthly revenue). Histogram — show distribution of one variable (age distribution). Scatter plot — show relationship between two numbers. Box plot — show distribution + outliers. Pie chart — show parts of a whole (max 5-6 slices).

Remember: Choose chart based on your data type and what you want to show.

Q34. How do you create a simple bar chart in Python?

Answer: Using Matplotlib: plt.bar(x_values, y_values). Using Seaborn: sns.barplot(data=df, x='category', y='value'). Always add title (plt.title()), x-label (plt.xlabel()), y-label (plt.ylabel()).

Example: sns.barplot(data=df, x='department', y='salary') — average salary by dept.

Q35. What is the difference between histogram and bar chart?

Answer: Bar chart is for categorical data (comparing groups). Histogram is for continuous numerical data (showing distribution — how many values fall in each range).

Remember: Bar chart = categories. Histogram = distribution of numbers.

Q36. How do you detect outliers visually?

Answer: Box plot — shows IQR and outliers as dots beyond the whiskers. Scatter plot — unusual points far from the main cluster. Histogram — bars far away from the main distribution.

Example: sns.boxplot(data=df, y='salary') — clearly shows outlier salaries.

SECTION 5 — Python General / OOPS Basics

Q37. What is a list comprehension?

Answer: A list comprehension is a short way to create a list using a single line of code instead of a for loop.

*Example: squares = [x**2 for x in range(10)] — creates [0,1,4,9,16,25,36,49,64,81]*

Remember: Square brackets + expression + for loop = list comprehension

Q38. What is the difference between a module and a library?

Answer: A module is a single Python file (.py) with functions and variables. A library (or package) is a collection of multiple modules. Example: pandas is a library made up of many modules.

Example: import pandas — imports the whole library. from pandas import DataFrame — imports just one part.

Q39. What is exception handling (try-except)?

Answer: Exception handling prevents your program from crashing when an error occurs. You put risky code in 'try' block. If error happens, 'except' block runs instead of crashing.

Example: try: result = 10/0 except ZeroDivisionError: print('Cannot divide by zero')

Q40. What is the difference between append() and extend() in a list?

Answer: append() adds one item to the end of a list. extend() adds all items from another list to the end.

Example: [1,2].append(3) = [1,2,3]. [1,2].extend([3,4]) = [1,2,3,4].

Q41. What is a Class and Object? (Basic OOP)

Answer: A class is a template/blueprint. An object is an instance created from that template. Think: Class = cookie cutter, Object = actual cookies made from it.

Example: class Student: def __init__(self,name): self.name=name / s = Student('Priya') — s is an object.

Q42. What does 'import' do in Python?

Answer: import brings in a library or module so you can use its functions. Without importing, you cannot use Pandas, NumPy, etc.

Example: import pandas as pd — import pandas and give it a short name 'pd'.

Q43. What is the difference between print() and return?

Answer: print() shows the value on screen but doesn't save it. return sends the value back from a function so it can be stored or used later.

Example: def add(a,b): return a+b — result = add(3,4) stores 7. If you use print instead, you can't store it.

Quick Reference — Most Used Commands

Command	What it does
`import pandas as pd`	Load Pandas library
`pd.read_csv('file.csv')`	Read CSV file
`df.head() / df.tail()`	First / last 5 rows
`df.shape`	(rows, columns)
`df.info()`	Column types + null counts
`df.describe()`	Statistics (mean, min, max...)
`df.isnull().sum()`	Count missing values
`df.dropna()`	Remove rows with missing values
`df.fillna(value)`	Fill missing values
`df[df['col'] > 10]`	Filter rows
`df['new_col'] = ...`	Add new column
`df.groupby('col').mean()`	Group and average
`pd.merge(df1,df2,on='id')`	Join two tables
`df.sort_values('col')`	Sort ascending
`df.drop_duplicates()`	Remove duplicate rows
`df.rename(columns={...})`	Rename columns
`df.astype(int)`	Change data type
`df.value_counts()`	Count each unique value
`df.corr()`	Correlation between columns

Keep coding every day — practice makes perfect! ■